Hugging Face AI Agents Course — Complete & Exhaustive Master Notes: Unit 1 (Introduction to Agents)

This document serves as a complete, standalone revision reference covering every single sub-unit, concept, architecture, table, and code implementation from **Unit 1: Introduction to Agents** of the Hugging Face AI Agents Course.

Module 1.1: What is an Agent?

1. The Big Picture Analogy: Alfred the Agent

To understand AI Agents intuitively, consider **Alfred**:

1. **Command (Natural Language Understanding)**: A user says, *"Alfred, I would like a coffee please."* Alfred parses and understands the natural language request.
2. **Reasoning & Planning**: Before taking action, Alfred plans the necessary sequence of steps and identifies required tools:
    - Step 1: Go to the kitchen.
    - Step 2: Use the coffee machine.
    - Step 3: Brew the coffee.
    - Step 4: Bring the coffee back.
3. **Action & Tool Use**: Alfred interacts with his environment using a tool from his known toolkit (activating the coffee machine).
4. **Goal Fulfillment**: Alfred brings the freshly brewed coffee back to the user.

2. Formal Definition of an AI Agent

> **Definition**: An **Agent** is a system that leverages an AI model to interact with its environment in order to achieve a user-defined objective. It combines **reasoning**, **planning**, and the **execution of actions** (often via external tools) to fulfill tasks.

It is called an *Agent* because it possesses **agency**—the capacity to act independently and interact with its environment.

3. The Two Main Parts of an Agent

- **The Brain (AI Model)**:
  - Where all thinking, reasoning, and planning happen.
  - Evaluates context and decides which **Actions** to take based on the current situation.
  - Typically a **Large Language Model (LLM)** (text-in, text-out, e.g., GPT-4, Llama 3, Gemini) or a **Vision Language Model (VLM)** if multimodal visual inputs are required.
- **The Body (Capabilities and Tools)**:
  - Represents everything the Agent is equipped to do in its environment.
  - Just as humans lack wings and cannot execute the `"fly"` action (but can `"walk"`, `"run"`, `"grab"`), an AI Agent's possible actions are strictly bounded by the **Tools** it has been equipped with.

Crucial Distinction — Actions vs. Tools: **Actions are NOT the same as Tools.** An Action is a high-level deliberate step taken by the agent, which may involve calling **multiple Tools** (or a single tool) to complete.

4. The Spectrum of "Agency" (From smolagents Conceptual Guide)

Agents do not exist as a binary "yes/no" system; they exist on a continuous spectrum of increasing agency based on how much control the LLM's output has over the program's execution flow:

-

Agency Level	Description	Name / Category	Example Code Pattern
☆☆☆ (Level 0)	Agent output has no impact on program flow	**Simple Processor**	`process_llm_output(llm_response)`
★☆☆ (Level 1)	Agent output determines basic control flow (branching)	**Router**	`if llm_decision(): path_a() else: path_b()`
★★☆ (Level 2)	Agent output determines function/tool execution & arguments	**Tool Caller**	`run_function(llm_chosen_tool, llm_chosen_args)`

Agency Level	Description	Name / Category	Example Code Pattern
★★★ (Level 3)	Agent output controls iteration, loop continuation, and multi-step execution	**Multi-step Agent**	`while llm_should_continue(): execute_next_step()`
★★★ (Level 4)	One agentic workflow can trigger/orchestrate another agentic workflow	**Multi-Agent**	`if llm_trigger(): execute_agent()`

5. Real-World Examples & Tasks

- **Personal Virtual Assistants (Siri, Alexa, Google Assistant)**: Analyze context, query databases, set reminders, send emails (e.g., calling `send_message_to("Manager", "Can we postpone today's meeting?")`), and control smart devices.
- **Customer Service Chatbots**: Answer queries, walk through troubleshooting, open support tickets in internal databases, complete transactions, and adapt to improve user satisfaction or conversion rates.
- **AI Non-Playable Characters (NPCs) in Video Games**: Replace rigid behavior trees with LLM-powered agents that respond contextually, adapt to unpredictable player interactions, and generate nuanced dialogue.

Module 1.2: What are LLMs? (The Brain of the Agent)

1. Foundation: The Transformer Architecture

Large Language Models (LLMs) are deep learning models trained on vast corpora of text to understand and generate human language, containing millions to billions (10^9) of parameters. Almost all modern LLMs are built on the **Transformer architecture** (introduced in *"Attention Is All You Need"*, popularized starting with Google's BERT in 2018).

2. The Three Types of Transformers

The original Transformer architecture consists of two primary components: an **Encoder** (left side) and a **Decoder** (right side). Variations yield three distinct types of Transformer models:

-

Transformer Type	How It Works	Examples	Primary Use Cases	Typical Parameter Size
1. Encoders	Takes text as input and outputs a dense vector representation (**embedding**) capturing semantic meaning.	**BERT** (Google), RoBERTa	Text classification, semantic search, Named Entity Recognition (NER)	Millions of parameters
2. Decoders	Focuses on generating new tokens to complete a sequence, **one token at a time** (autoregressive).	**Llama 3** (Meta), **GPT-4** (OpenAI), **DeepSeek-R1**, **SmolLM2** (HF), **Gemma** (Google), **Mistral**	Text generation, chatbots, code generation, **AI Agents**	Billions (10^9) of parameters
3. Seq2Seq (Encoder–Decoder)	Combines both: the **Encoder** first processes the input sequence into a context representation, then the **Decoder** generates an output sequence.	**T5**, **BART**	Translation, Summarization, Paraphrasing	Millions of parameters

Note: Modern LLMs powering AI Agents are almost exclusively **Decoder-based Transformers** with billions of parameters.

3. Tokenization & Sub-word Units

- **Objective of an LLM**: Predict the **next token** given a sequence of previous tokens.
- **What is a Token?**: The fundamental unit of information an LLM processes. Rather than whole words (English has ~600,000 words), LLMs use **sub-word tokenization** for efficiency (e.g., Llama 2 has a vocabulary size of ~32,000 tokens).
- **Sub-word Combination Example**: The sub-word tokens `"interest"` + `"ing"` combine to form `"interesting"`, or `"ed"` is appended to form `"interested"`.

4. Special Tokens & End of Sequence (EOS)

LLMs use model-specific **special tokens** to open and close structured components of generation (start/end of sequence, message turns, or responses). The most critical special token is the **EOS (End of Sequence)** token, which signals the LLM to stop generating tokens.

-

Model	Provider	EOS Token	Functionality
GPT-4	OpenAI	`<|endoftext|>`	End of message text
Llama 3	Meta	`<|eot_id|>`	End of sequence / turn
DeepSeek-R1	DeepSeek	`<|end_of_sentence|>`	End of message text
SmolLM2	Hugging Face	`<|im_end|>`	End of instruction or message
Gemma	Google	`<end_of_turn>`	End of conversation turn

(Special tokens can always be inspected inside a model's tokenizer_config.json on the Hugging Face Hub).

5. Autoregressive Next-Token Prediction & Decoding Strategies

- **Autoregressive Generation**: The output from one forward pass is appended to the input and fed back into the model as the input for the next pass. This loop repeats token-by-token until the model predicts the **EOS token**.
- **Single Decoding Loop Step**:
    - 1. Input text is tokenized into token IDs.
    - 2. Model computes a sequence representation capturing both semantic meaning and **positional information** of each token.
    - 3. Model outputs raw **scores (logits)** ranking the likelihood of every token in its vocabulary being the next token.
- **Decoding Strategies (Selecting the Next Token)**:
    - **Greedy Decoding (Maximum Score)**: The simplest strategy—always selecting the single token with the highest probability score at each step.
    - **Beam Search**: Explores multiple candidate sequences simultaneously (`num_beams`) to find the sequence with the **maximum total cumulative score**, even if some individual intermediate tokens had lower scores.

        - **Key Beam Search Parameters**:
            - `inputs`: Input prompt sequence.
            - `max_new_tokens`: Number of decoding steps / tokens to generate.
            - `num_beams`: Number of parallel candidate beams tracked.
            - `length_penalty`: Adjusts preference for sequence length (`> 0.0` promotes longer sequences; `< 0.0` encourages shorter sequences; does not change beam paths, only final sequence selection).
            - `num_return_sequences`: Number of top candidate sequences returned at the end (`<= num_beams`).

6. Attention Mechanism & Context Length

- **Attention ("Attention Is All You Need")**: When predicting the next token, not every word in the prompt carries equal weight. In *"The capital of France is..."*, the words **"France"** and **"capital"** carry the highest weight. The **Attention mechanism** dynamically identifies and weights the most relevant past tokens.
- **Context Length (Attention Span)**: The maximum number of tokens an LLM can process at once in its input sequence and attend over.
- **Prompting Importance**: Because an LLM's sole task is predicting the next token by attending over input tokens, the exact wording and structure of the input sequence (**the prompt**) heavily governs output quality.

7. How LLMs Are Trained & Used

- **1. Pre-training (Self-Supervised / Masked Language Modeling)**: Trained on massive unlabeled text datasets to predict the next token (or masked tokens). The model learns grammar, language structure, world facts, and reasoning patterns without human labels.
- **2. Fine-tuning (Supervised Learning)**: Post-pretraining phase where the base model is trained on curated datasets for specific objectives (e.g., following conversational instructions, tool calling, code generation, or classification).
- **3. Deployment Options**:
    - **Run Locally**: Requires sufficient local GPU/VRAM hardware (using `transformers`, `vLLM`, `ollama`).

    - **Cloud / Serverless API**: Running inference via remote APIs such as the **Hugging Face Serverless Inference API** (requires an `HF_TOKEN`).

Module 1.3: Messages, Special Tokens, and Chat Templates

1. The UI Abstraction vs. Concatenated Prompt Reality

When chatting with ChatGPT or HuggingChat, it appears as an ongoing back-and-forth exchange of separate messages. **This is a UI abstraction.**

- **Stateless Memory**: LLMs do **not** inherently remember past turns of a conversation.
- **Full Concatenation**: On every single turn, all messages in the conversation history (system + user + assistant turns) are concatenated and formatted by a **Chat Template** into one giant stand-alone string prompt and fed to the LLM in full.

2. Message Types & Roles

- **System Messages (`role: "system"`)**:
    - Persistent instructions defining how the model should behave, its persona (e.g., Polite Alfred vs. Rebel Alfred), and operational rules.
    - **In AI Agents**, the System Message is where we inject:
        - 1. **Available Tools** (their names, descriptions, and argument schemas).
        - 2. **Action Formatting Instructions** (how to output JSON or Python code).
        - 3. **Thought-Action-Observation Cycle rules** (how to segment reasoning).
- **User (`role: "user"`)** & Assistant (`role: "assistant"`) Messages:
    - Alternating turns between the human (`user`) and the LLM (`assistant`) that preserve multi-turn context.

3. Base Models vs. Instruct Models

- **Base Model** (e.g., `SmolLM2-135M`): Trained solely on raw text to predict the next token. Given a question, it might just continue generating more questions rather than answering.
- **Instruct Model** (e.g., `SmolLM2-135M-Instruct`): Fine-tuned specifically to follow instructions and engage in structured multi-turn conversations using a specific **Chat Template** (such as **ChatML**).

4. Chat Templates & Formatting Variations Across Models

Different Instruct models expect completely different special tokens and role delimiters. **Chat Templates** (written in **Jinja2** inside the Hugging Face tokenizer) bridge the gap by converting a standard Python list of ChatML dictionaries into the exact raw prompt string the target model was trained on.

Example Conversation List (ChatML format):

-

```python
conversation = [
    {"role": "user", "content": "I need help with my order"},
    {"role": "assistant", "content": "I'd be happy to help. Could you provide your order number?"},
    {"role": "user", "content": "It's ORDER-123"},
]
```

How SmolLM2 Formats This Conversation (ChatML Style):

-

```
<|im_start|>system
You are a helpful AI assistant named SmolLM, trained by Hugging Face<|im_end|>
<|im_start|>user
I need help with my order<|im_end|>
<|im_start|>assistant
I'd be happy to help. Could you provide your order number?<|im_end|>
<|im_start|>user
It's ORDER-123<|im_end|>
<|im_start|>assistant
```

How Llama 3.2 Formats the Exact Same Conversation:

-

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 10 Feb 2025

<|eot_id|><|start_header_id|>user<|end_header_id|>
```

```
I need help with my order<|eot_id|><|start_header_id|>assistant<|end_header_id|>

I'd be happy to help. Could you provide your order
number?<|eot_id|><|start_header_id|>user<|end_header_id|>

It's ORDER-123<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

5. Inside a Jinja2 Chat Template (SmolLM2-135M-Instruct)

- **Code Agents**: The model generates executable Python code blocks directly. Supported prominently by frameworks like `smolagents`, code agents execute control flow, variable manipulation, and multiple tool calls directly within a secure Python runtime, drastically reducing round-trip token overhead.

```
None
{% for message in messages %}
{% if loop.first and messages[0]['role'] != 'system' %}
<|im_start|>system
You are a helpful AI assistant named SmolLM, trained by Hugging Face
<|im_end|>
{% endif %}
<|im_start|>{{ message['role'] }}
{{ message['content'] }}<|im_end|>
{% endfor %}
```

6. Applying Chat Templates in Python (apply_chat_template)

```
Python
from transformers import AutoTokenizer

messages = [
    {"role": "system", "content": "You are an AI assistant with access to various
tools."},
```

```
    {"role": "user", "content": "Hi !"},
    {"role": "assistant", "content": "Hi human, what can help you with ?"},
]

tokenizer = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM2-1.7B-Instruct")
rendered_prompt = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True
)
```

Module 1.4: What are Tools?

1. Definition & Purpose of AI Tools

Definition: A **Tool** is a function given to the LLM that fulfills a clear, specific objective.

Why Tools Are Essential:

- **1. Overcoming Static Training Cutoffs**: LLMs only know facts prior to their training cutoff. Without a tool, asking for today's weather causes the LLM to hallucinate.
- **2. Complementing LLM Weaknesses**: LLMs struggle with exact arithmetic or deterministic logic; giving them a `calculator` or Python interpreter yields exact results.
- **3. Interacting with the External World**: Fetching live web pages, generating images, querying databases, or calling external APIs.

Common Examples of Tools in AI Agents:

Tool Type	Description
Web Search	Fetches up-to-date information from the internet.
Image Generation	Creates images based on text descriptions.
Retrieval	Retrieves information from external knowledge bases / vector stores.

Tool Type	Description
API Interface	Interacts with external services (GitHub, YouTube, Spotify, weather APIs).

2. Core Components of a Tool

Every Tool must contain four essential elements:

- 1. **A textual description** of what the function does (so the LLM knows when to use it).
- 2. **A Callable** (the underlying Python function or executable logic).
- 3. **Arguments with typings** (expected input parameters and their data types).
- 4. **(Optional) Outputs with typings** (expected return type(s)).

3. How Tools Work Behind the Scenes

LLMs **only** take text as input and output text—they cannot execute functions themselves:

- **1. System Prompt Injection**: We inject the textual descriptions of all available tools into the **System Prompt**.
- **2. Textual Tool Invocation**: When the user asks for the weather in Paris, the LLM outputs structured text representing a tool call (e.g., `call weather_tool('Paris')` or a JSON blob).
- **3. Background Execution by the Agent Runtime**: The Agent intercepts this text, executes the actual Python function on the LLM's behalf, and gets the real data.
- **4. Appending & Responding**: The Agent appends the tool's output as a new message/observation into the conversation and passes the updated prompt back to the LLM, which then writes a natural response to the user.

4. Implementing a Generic Tool Class & @tool Decorator from Scratch

Instead of manually formatting string descriptions like `"Tool Name: calculator, Description: Multiply two integers., Arguments: a: int, b: int, Outputs: int"` (which is fragile and error-prone), we use Python's `inspect` module and type hints to auto-generate tool descriptions via a `@tool` decorator:

```python
import inspect
from typing import Callable
```

```
class Tool:
    """
    A class representing a reusable piece of code (Tool).
    """
    def __init__(self, name: str, description: str, func: Callable, arguments:
list, outputs: str):
        self.name = name
        self.description = description
        self.func = func
        self.arguments = arguments
        self.outputs = outputs

    def to_string(self) -> str:
        """Return a string representation of the tool for the LLM system
prompt."""
        args_str = ", ".join([
            f"{arg_name}: {arg_type}" for arg_name, arg_type in self.arguments
        ])
        return (
            f"Tool Name: {self.name},"
            f" Description: {self.description},"
            f" Arguments: {args_str},"
            f" Outputs: {self.outputs}"
        )

    def __call__(self, *args, **kwargs):
        """Invoke the underlying callable function."""
        return self.func(*args, **kwargs)


def tool(func):
    """
    A decorator that inspects a Python function's signature, type annotations,
    and docstring to automatically return a Tool instance.
    """
    signature = inspect.signature(func)
```

```
    # Extract (param_name, param_annotation) pairs for inputs
    arguments = []
    for param in signature.parameters.values():
        annotation_name = (
            param.annotation.__name__
            if hasattr(param.annotation, '__name__')
            else str(param.annotation)
        )
        arguments.append((param.name, annotation_name))

    # Determine the return annotation
    return_annotation = signature.return_annotation
    if return_annotation is inspect._empty:
        outputs = "No return annotation"
    else:
        outputs = (
            return_annotation.__name__
            if hasattr(return_annotation, '__name__')
            else str(return_annotation)
        )

    description = func.__doc__ or "No description provided."
    name = func.__name__

    return Tool(
        name=name,
        description=description,
        func=func,
        arguments=arguments,
        outputs=outputs
    )

# Usage Example:
@tool
def calculator(a: int, b: int) -> int:
    """Multiply two integers."""
```

```
    return a * b

print(calculator.to_string())
# Output: Tool Name: calculator, Description: Multiply two integers., Arguments:
a: int, b: int, Outputs: int
```

5. Model Context Protocol (MCP): A Unified Tool Interface

Model Context Protocol (MCP) is an open standard that standardizes how applications provide tools and context to LLMs.

- **Key Benefits of MCP**:
    - 1. A growing ecosystem of pre-built integrations that an LLM can directly plug into.
    - 2. Flexibility to switch seamlessly between different LLM providers and vendors.
    - 3. Standardized best practices for securing data within your infrastructure.
    - 4. Eliminates the need to rewrite custom tool wrappers for every single agent framework.

Module 1.5: Understanding AI Agents through the Thought–Action–Observation Cycle

1. The Continuous while Loop Analogy

Agents operate in an iterative cycle (analogous to a programming `while` loop that runs until the objective is fulfilled):

Thought → Action → Observation → Updated Thought → ... → Final Answer

- **Thought**: The LLM part of the Agent analyzes context and decides what the next step should be.
- **Action**: The Agent takes action by calling the appropriate tool(s) with associated arguments.
- **Observation**: The model receives and reflects on the raw response/feedback returned by the tool.

(All three rules—Agent behavior, available Tools, and the Thought-Action-Observation format—are baked directly into the System Prompt).

2. Step-by-Step Walkthrough: Alfred the Weather Agent

User asks: *"What's the current weather in New York?"*

- **1. Thought (Internal Reasoning)**: *"The user needs current weather information for New York. I have access to a tool that fetches weather data. First, I need to call the weather API to get up-to-date details."*
- **2. Action (Tool Usage)**: Alfred emits a structured JSON command:

```JSON
{
  "action": "get_weather",
  "action_input": { "location": "New York" }
}
```

- **3. Observation (Feedback from Environment)**: The tool executes and returns raw data appended to the prompt: *"Current weather in New York: partly cloudy, 15°C, 60% humidity."*
- **4. Updated Thought (Reflecting)**: *"Now that I have the weather data for New York, I can compile an answer for the user."*
- **5. Final Action**: Emits `"Final answer: The current weather in New York is partly cloudy with a temperature of 15°C and 60% humidity."`, closing the loop. *(If the observation had returned an API error, Alfred would re-enter the loop to retry or adjust strategy).*

Module 1.6: Thought — Internal Reasoning and the ReAct Approach

1. What Are Thoughts?

Thoughts represent the Agent's **inner monologue**—its internal reasoning and planning process in the prompt before taking an action. They allow the agent to decompose complex problems into smaller steps, incorporate past observations, and adapt plans.

2. The 8 Common Thought Types

Type of Thought	Concrete Example
1. Planning	*"I need to break this task into three steps: 1) gather data, 2) analyze trends, 3) generate report"*
2. Analysis	*"Based on the error message, the issue appears to be with the database connection parameters"*
3. Decision Making	*"Given the user's budget constraints, I should recommend the mid-tier option"*
4. Problem Solving	*"To optimize this code, I should first profile it to identify bottlenecks"*
5. Memory Integration	*"The user mentioned their preference for Python earlier, so I'll provide examples in Python"*
6. Self-Reflection	*"My last approach didn't work well, I should try a different strategy"*
7. Goal Setting	*"To complete this task, I need to first establish the acceptance criteria"*
8. Prioritization	*"The security vulnerability should be addressed before adding new features"*

(Note: For LLMs specifically fine-tuned for function-calling, emitting an explicit Thought step can be optional).

3. Chain-of-Thought (CoT) vs. ReAct (Reasoning + Acting)

- **Chain-of-Thought (CoT)**:
  - A pure prompting technique (typically triggered by *"Let's think step by step"*) that guides the model to reason sequentially through logical or mathematical problems **internally**, **without** using external tools.
  - *Example*: `Question: What is 15% of 200? -> Thought: Let's think step by step. 10% of 200 is 20, and 5% of 200 is 10, so 15% is 30. -> Answer: 30`
- **ReAct (Reasoning + Acting)**:

- A prompting technique that interleaves step-by-step reasoning (**Thought**) with external tool execution (**Action**) and environmental feedback (**Observation**).
- *Example*: `Thought: I need to find the latest weather in Paris. -> Action: Search["weather in Paris"] -> Observation: It's 18°C and cloudy. -> Thought: Now that I know the weather... -> Action: Finish["It's 18°C and cloudy in Paris."]`

Feature	Chain-of-Thought (CoT)	ReAct (Reasoning + Acting)
Step-by-step logic	✅ Yes	✅ Yes
External tools	❌ No	✅ Yes (Actions + Observations)
Best suited for	Logic, math, self-contained internal tasks	Information-seeking, dynamic multi-step tasks

4. Training-Level Reasoning Models (<think> tokens)

Recent reasoning models like **DeepSeek-R1** and **OpenAI's o1** are fine-tuned at the training level (via Reinforcement Learning) to think before answering, using explicit special tokens like `<think>` and `</think>` to separate their internal reasoning phase from the final answer—distinct from CoT/ReAct which are prompting strategies.

Module 1.7: Actions — Enabling the Agent to Engage with Its Environment

1. Types of Agents by Action Format

Type of Agent	How Actions Are Specified
1. JSON Agent	The Action to take and its arguments are specified in a structured **JSON format**.
2. Code Agent	The Agent writes an **executable code block** (typically Python) that is interpreted externally.

Type of Agent	How Actions Are Specified
3. Function-calling Agent	A subcategory of the JSON Agent where the model has been specifically fine-tuned to generate a dedicated tool-call message for each action.

2. Core Purposes of Actions

- **Information Gathering**: Performing web searches, querying databases, retrieving documents.
- **Tool Usage**: Making API calls, running calculations, executing code.
- **Environment Interaction**: Manipulating digital interfaces or controlling physical hardware/devices.
- **Communication**: Engaging with users via chat or collaborating with other agents.

3. The "Stop and Parse" Approach (Critical Execution Mechanism)

Because an LLM only generates text, it will keep generating tokens indefinitely (including hallucinating fake tool outputs!) unless halted. The **Stop and Parse** approach enforces reliable tool execution in three steps:

- **1. Generation in a Structured Format**: The agent outputs its intended action in a predetermined format (JSON or code block).
- **2. Halting Further Generation (STOP)**: As soon as the action definition ends (e.g., right before the word `"Observation:"` or closing code block), the LLM is forced to **STOP generating new tokens**. This passes control back from the LLM to the Agent framework.
- **3. Parsing the Output (PARSE)**: An external parser reads the emitted text, identifies which Tool to call, and extracts the required arguments.

4. Code Agents vs. JSON Agents

Instead of emitting a flat JSON blob, a **Code Agent** (like `smolagents`' `CodeAgent`) generates an executable Python snippet:

```python
# Code Agent Example: Retrieve Weather Information
def get_weather(city):
```

```
    import requests
    api_url = f"https://api.weather.com/v1/location/{city}?apiKey=YOUR_API_KEY"
    response = requests.get(api_url)
    if response.status_code == 200:
        data = response.json()
        return data.get("weather", "No weather information available")
    else:
        return "Error: Unable to fetch weather data."

result = get_weather("New York")
final_answer = f"The current weather in New York is: {result}"
print(final_answer)
```

4 Key Advantages of Code Agents:

- **1. Expressiveness**: Code naturally represents loops, conditionals (`if/else`), and nested function compositions that are clumsy or impossible in flat JSON.
- **2. Modularity and Reusability**: Generated code can define helper functions and reuse variables across steps.
- **3. Enhanced Debuggability**: Programming syntax errors yield clear tracebacks that are easier to detect and self-correct.
- **4. Direct Integration**: Can directly import and use external libraries and APIs for data processing.

(Security Note: Executing LLM-generated code carries risks like prompt injection or malicious execution, so always use frameworks like smolagents with built-in sandboxing safeguards).

Module 1.8: Observe — Integrating Feedback to Reflect and Adapt

1. What Are Observations & The 3-Step Phase

Observations are the signals from the environment (tool logs, API responses, error tracebacks) that let the Agent perceive the consequences of its actions. During the Observation phase, the agent:

- **1. Collects Feedback**: Receives data or confirmation of whether its action succeeded or failed.
- **2. Appends Results**: Integrates the new information at the end of the prompt context, effectively updating its short-term memory.

- **3. Adapts its Strategy**: Uses the updated context in the next Thought step to refine subsequent actions.

2. The 5 Types of Observations

Type of Observation	Concrete Examples
1. System Feedback	Error messages, exceptions, success notifications, HTTP status codes
2. Data Changes	Database updates, file system modifications, state changes
3. Environmental Data	Sensor readings, system metrics, resource usage
4. Response Analysis	API responses, search query results, computation outputs
5. Time-based Events	Deadlines reached, scheduled tasks completed

3. How Results Are Appended by the Framework

After the LLM emits an Action and halts, the framework executes three sequential steps:

- 1. **Parse** the action to identify the function(s) to call and the argument(s) to pass.
- 2. **Execute** the action in the environment.
- 3. **Append** the returned result to the conversation history as an `Observation: ...` block before re-invoking the LLM.

Module 1.9: Dummy Agent Library (Building an Agent from Scratch in Python)

To understand what agent frameworks do under the hood without framework magic, we build a complete **Dummy Agent from scratch** using only Python and the `huggingface_hub InferenceClient` (Serverless API).

1. Setting up the Serverless API (InferenceClient)

```python
import os
```

```python
from huggingface_hub import InferenceClient

HF_TOKEN = os.environ.get("HF_TOKEN")
client = InferenceClient(model="moonshotai/Kimi-K2.5", token=HF_TOKEN)
```

2. Constructing the Agent System Prompt

The core of any agent library is appending tool descriptions and Thought-Action-Observation formatting rules into the `SYSTEM_PROMPT`:

Python

```
SYSTEM_PROMPT = """Answer the following questions as best you can. You have access to the following tools:

get_weather: Get the current weather in a given location

The way you use the tools is by specifying a json blob.
Specifically, this json should have an `action` key (with the name of the tool to use) and an `action_input` key (with the input to the tool going here).

The only values that should be in the "action" field are:
get_weather: Get the current weather in a given location, args: {"location": {"type": "string"}}
example use :
{{
  "action": "get_weather",
  "action_input": {"location": "New York"}
}}

ALWAYS use the following format:

Question: the input question you must answer
Thought: you should always think about one action to take. Only one action at a time in this format:
Action:
```
$JSON_BLOB
```
```

```
Observation: the result of the action. This Observation is unique, complete, and the source of truth.
... (this Thought/Action/Observation can repeat N times, you should take several steps when needed. The $JSON_BLOB must be formatted as markdown and only use a SINGLE action at a time.)

You must always end your output with the following format:

Thought: I now know the final answer
Final Answer: the final answer to the original input question

Now begin! Reminder to ALWAYS use the exact characters `Final Answer:` when you provide a definitive answer. """
```

3. The Hallucinated Observation Problem & Fixing It with stop=["Observation:"]

- **Without a Stop Sequence**: If we call `client.chat.completions.create(messages=messages)` directly, the LLM generates the `Thought:`, the `Action:` JSON blob, **and then continues generating a fabricated Observation:** (e.g., hallucinating *"The current weather in London is partly cloudy with a temperature of 12°C"*) without our Python function ever running!
- **The Fix (stop=["Observation:"])**: We pass `stop=["Observation:"]` so the LLM halts immediately after emitting the `Action:` JSON block:

Python
```python
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": "What's the weather in London?"},
]

# Step 1: Generate Thought and Action, stopping right before "Observation:"
output = client.chat.completions.create(
    messages=messages,
    max_tokens=150,
```

```
    stop=["Observation:"], # CRITICAL: Halts generation before hallucinating
Observation!
    extra_body={'thinking': {'type': 'disabled'}},
)

# Step 2: Execute the actual Python tool function
def get_weather(location):
    return f"the weather in {location} is sunny with low temperatures. \n"

# Step 3: Concatenate the assistant's Thought+Action with the real Observation and
resume generation
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": "What's the weather in London ?"},
    {"role": "assistant", "content": output.choices[0].message.content +
"Observation:\n" + get_weather('London')},
]

final_output = client.chat.completions.create(
    messages=messages,
    stream=False,
    max_tokens=200,
    extra_body={'thinking': {'type': 'disabled'}},
)
print(final_output.choices[0].message.content)
# Output: Final Answer: The weather in London is sunny with low temperatures.
```

Module 1.10: Let's Create Our First Agent Using smolagents

1. What is smolagents?

`smolagents` is Hugging Face's lightweight framework designed to abstract away the tedious manual prompt concatenation, stop-token management, and parsing we did in the Dummy Agent section. It focuses heavily on **`CodeAgent`**—agents that write and execute Python code blocks as their Actions.

2. Requirements for Defining Tools in smolagents

When using the `@tool` decorator in `smolagents`, two rules are mandatory:

- **1. Strict Type Hints**: You **must** provide input type hints and a return type hint (e.g., `def get_current_time_in_timezone(timezone: str) -> str:`).
- **2. Formatted Docstring with Args:**: You **must** include a clear docstring describing the tool AND an `Args:` section providing a textual description for every single argument.

3. Complete app.py Reference Implementation

Below is the complete `smolagents` agent setup from Unit 1 (using `Qwen/Qwen2.5-Coder-32B-Instruct` via `InferenceClientModel`, custom `@tool` functions, Hub-loaded tools, and `prompts.yaml`):

```python
from smolagents import CodeAgent, DuckDuckGoSearchTool, InferenceClientModel,
load_tool, tool
import datetime
import requests
import pytz
import yaml
from tools.final_answer import FinalAnswerTool
from Gradio_UI import GradioUI

# 1. Custom Tool Template (with mandatory type hints and Args docstring)
@tool
def my_custom_tool(arg1: str, arg2: int) -> str:
    """A tool that does nothing yet
    Args:
        arg1: the first argument
        arg2: the second argument
    """
    return "What magic will you build ?"

# 2. Working Timezone Tool
@tool
def get_current_time_in_timezone(timezone: str) -> str:
    """A tool that fetches the current local time in a specified timezone.
    Args:
```

```
        timezone: A string representing a valid timezone (e.g.,
'America/New_York').
    """
    try:
        tz = pytz.timezone(timezone)
        local_time = datetime.datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
        return f"The current local time in {timezone} is: {local_time}"
    except Exception as e:
        return f"Error fetching time for timezone '{timezone}': {str(e)}"

final_answer = FinalAnswerTool()

# 3. Configure the LLM Engine via Serverless API
model = InferenceClientModel(
    max_tokens=2096,
    temperature=0.5,
    model_id='Qwen/Qwen2.5-Coder-32B-Instruct',
    custom_role_conversions=None,
)

# 4. Load Pre-built Tool from the Hugging Face Hub & Built-in Search Tool
image_generation_tool = load_tool("agents-course/text-to-image",
trust_remote_code=True)
search_tool = DuckDuckGoSearchTool()

# 5. Load System Prompt Templates from external YAML file
with open("prompts.yaml", 'r') as stream:
    prompt_templates = yaml.safe_load(stream)

# 6. Initialize the CodeAgent
agent = CodeAgent(
    model=model,
    tools=[final_answer, get_current_time_in_timezone, search_tool,
image_generation_tool],
    max_steps=6,
    verbosity_level=1,
    grammar=None,
```

```
    planning_interval=None,
    name=None,
    description=None,
    prompt_templates=prompt_templates
)

# 7. Launch Interactive Gradio UI (for Hugging Face Spaces)
GradioUI(agent).launch()
```

Module 1.11: Quick Revision Cheat Sheet (Unit 1 Quiz & Certification Prep)

- **Agent Definition**: AI model (Brain) + Capabilities/Tools (Body) interacting with an environment via reasoning, planning, and action execution.
- **Agency Spectrum**: Simple Processor (☆☆☆) → Router (★☆☆) → Tool Caller (★★☆) → Multi-step Agent (★★★) → Multi-Agent (★★★).
- **3 Transformer Types**:
  - *Encoder* (BERT — embeddings, classification, semantic search, millions of params).
  - *Decoder* (Llama, GPT-4, DeepSeek-R1, SmolLM2 — autoregressive next-token prediction, billions of params, powers Agents).
  - *Seq2Seq / Encoder-Decoder* (T5, BART — translation, summarization).
- **Decoding**: Autoregressive loop runs until the model emits its specific **EOS token** (`<|endoftext|>`, `<|eot_id|>`, `<|im_end|>`, `<end_of_turn>`). Strategies include **Greedy** (max score) and **Beam Search** (`num_beams`, `length_penalty`).
- **Chat Templates**: LLMs are stateless; every turn concatenates the full message history using a Jinja2 `chat_template` (`tokenizer.apply_chat_template`).
- **Tools & MCP**: Functions described in the System Prompt via name, docstring description, typed arguments, and return types. **MCP (Model Context Protocol)** standardizes tool interfaces across providers.
- **CoT vs. ReAct**: CoT is internal step-by-step reasoning *without* external tools; ReAct interleaves **Thought → Action → Observation** *with* external tools.

- **Stop & Parse**: Halting LLM generation (`stop=["Observation:"]`) right after it outputs an Action prevents the LLM from hallucinating fake tool observations and allows the runtime parser to execute the real function.